TECHNICAL PORTFOLIO · SUMMER 2027 SOFTWARE ENGINEERING

Ishaan Chaturvedi

B.S. Computer Science, Minor in Mathematics · Georgia Tech · GPA 3.8 · Expected May 2028
ichat2006@gmail.com · github.com/ichat2006 · linkedin.com/in/ishaan-chaturvedi

Two deep dives

This packet goes deep on two systems I built rather than broad on many. Both are about the same underlying problem: deciding when real-world sensor data can be trusted, when it is late, missing, or possibly fake. That problem shows up in robot autonomy, and it shows up in curating the data that trains AI models.

Project	The hard part	Page
Fusion in Motion Rust GPS/IMU EKF + camera/LiDAR tracking (GT Cloud Robotics VIP)	A drift-invariant consistency check for GPS returning after a 40 s outage; a bounded queue with class-share guarantees; tracing a 3.5x tracking error to delivery-time stamping; avoiding innovation-gate lockout.	2
TrueTake Robot-training video review (Python, OpenCV, CLIP, MediaPipe, Claude)	Recovering the clock offset between a phone's video and gyroscope with normalized cross-correlation, using it to prove footage is genuine and to localize splices.	after Fusion in Motion

All numbers come from my own runs and are labeled with their scope (seeds, scenario). Where a result is single-seed or still needs a controlled experiment, the text says so.

Also built

- **Moog Space and Defense Group, Software Engineering Intern (Jun–Aug 2026):** rebuilt maintainable C source for an undocumented legacy binary through black-box testing and fixed its partial-read TCP message framing; built an event-driven WebSocket launcher service enforcing one active operator session; automated RHEL provisioning and hardening with Ansible, from up to 4 hours to about 10–15 minutes (roughly 95% less time per machine) at a 98% STIG pass rate, adopted by 2 more offices.
- **MALD & SWAD:** a Jetson Nano assistive-navigation device fusing YOLO detections with LiDAR depth in a 3-thread condition-variable pipeline, speaking warnings through an iOS app in about 60–130 ms end to end (200 ms target); field-tested with 6 users, 8 devices sold.
- **Metron Finance:** team fintech platform where I worked across the stack, mostly backend, and built the FastAPI ML service (9 REST endpoints, XGBoost scorer, PyTorch LSTM forecaster, online retraining, fallback when a model fails).

GT CLOUD ROBOTICS VIP · RUST · SEP 2026 – PRESENT

Fusion in Motion

Making a robot's GPS/IMU state estimator and object tracker degrade gracefully when the sensor stream stops being clean: overload, late data, and a 40-second GPS outage.

9% / 8%	3.5x	75.9%	~3,900
lower localization / tracking error under overload (3 seeds)	tracking-error bug traced to delivery-time stamping	lower return-window error after a 40 s outage	lines added on my branch, with tests

What I built, in one paragraph

Fusion in Motion is the GT Cloud Robotics VIP's Rust localization and tracking engine (PI: Ethan Boos): a GPS/IMU extended Kalman filter with IMU-bias estimation, plus camera/LiDAR multi-object tracking, driven by a seeded simulator and checked against regression baselines. My branch, `ishaan-sensor-pipeline`, is three commits and about 3,900 added lines across the estimator, a new pipeline module, tests, experiments, and docs. It answers one question: **what should a robot's state estimator do when its sensor stream stops being clean?** That covers three failure modes: more data than the worker can process, data that arrives late, and a sensor that disappears for 40 seconds and then comes back.

- **Sensors (simulated):** IMU at 100 Hz, GPS at 2 Hz (0.25 m σ), camera at 10 Hz (bearing only), LiDAR at 5 Hz (range + bearing).
- **Two clocks on every packet:** `measurement_time_ns` (when the sensor captured it) and `arrival_time_ns` (when the software got it). Most of the bugs below live in the gap between those two numbers.
- **What I added:** a bounded sensor queue with 5 drop policies and per-sensor expiry, a GPS-health monitor, GPS reacquisition logic (variance ramp plus a drift-invariant consistency check), freshness diagnostics, end-to-end and unit tests, regression fixtures, a run-analysis script, and the experiment write-ups.

1. Overload: a bounded queue that degrades on purpose

A real robot's compute can't always keep up with its sensors. I wrote `pipeline::replay`, a deterministic single-worker load model: every packet arrives at its timestamp, waits in a bounded FIFO, and costs a configurable service time per sensor type. When the queue is full, a policy decides who loses. Both the estimator and the tracker then consume only what was actually delivered, with delivery delays applied, so a policy's cost shows up in localization and tracking error rather than just in a drop counter.

The five policies are `drop_newest`, `drop_oldest`, `keep_latest`, `prefer_navigation` (evict camera/LiDAR before IMU/GPS), and `balanced`. Balanced is the interesting one. It guarantees each class (navigation vs. perception) a minimum share of the queue:

```
min_class = min( max( ceil(C · p / 100), C ≥ 4 ? 2 : 1 ), floor(C / 2) )      // p = 25 by
default
max_class = C − min_class
```

On overflow, if the incoming packet's class is under its cap, the victim comes from the other class: specifically the most over-represented *sensor* in that class, and never one that would leave a sensor type with zero queued packets (when C ≥ 4). If the other class is already at its floor, the incoming packet is dropped instead. If the incoming class is at its cap, it evicts the oldest queued packet from its own sensor (or its own class). FIFO order is preserved for everything that stays. Each sensor also gets an optional maximum queue wait, so a stale packet is expired rather than served late.

3-seed sweep, 15 s overload case	balanced	drop_newest
Ego localization RMSE (capacity 8)	**0.185 m**	0.204 m
Track RMSE, estimated ego (capacity 8)	**0.240 m**	0.262 m
Track RMSE, truth ego (capacity 8)	**0.026 m**	0.031 m
Track RMSE, truth ego (capacity 32)	**0.026 m**	0.033 m (prefer_navigation: 0.087 m)
What balanced gave up	197 IMU packets per run at capacity 8; zero GPS, camera, or LiDAR	

That's a 9% localization and 8% tracking improvement from shedding the one stream that is most redundant at 100 Hz. The lesson I wrote into the docs: `prefer_navigation` sounds safe, but starving perception made tracking more than 3x worse than balanced at capacity 32.

---

2. The 3.5x bug: late data stamped as new data

When I ran the queue against the GPS-recovery scenario, localization barely moved, but object-tracking error jumped from **0.035 m to 0.123 m (3.5x)**. The queue had dropped 748 of 6,000 IMU packets (12.5%) and no detections, so data loss wasn't the cause.

The cause was time. A camera or LiDAR detection that waited in the queue was being handed to the tracker stamped with its *delivery* time. The tracker then predicted each object forward to the wrong instant before associating and updating, so every update applied an old observation as if it were current. For a target moving at speed v with queue delay d, that injects a position bias on the order of $v \cdot d$ into every update. That is the classic out-of-sequence-measurement problem, and it is invisible in drop statistics.

The engine already had a tracker timing-compensation option that keeps each detection's capture time, but it was off by default, so nothing had ever exercised it under queue delay. Turning it on returned track error to **0.035 m** with no false-track samples. I made it the default for the overload scenario and documented why the GPS-recovery experiment itself stays queue-free when isolating the filter. The

general rule I took from it: **a queue changes when data arrives, never when it happened**, and every consumer must be explicit about which of the two clocks it is using.

---

3. A 40-second GPS outage, and the gate that locks GPS out

The recovery challenge drives the vehicle through a left and then right turn with GPS absent from 10 s to 50 s, leaving 10 s to recover. During the outage the filter dead-reckons on biased IMU data (simulated accelerometer bias 0.025 m/s², gyro bias 0.003 rad/s), so position error grows to meters (outage-window RMSE 3.06 m, about 21.5x the continuous-GPS control).

The trap. The EKF rejects any GPS fix whose normalized innovation exceeds a 3-σ gate. If the filter's covariance grows more slowly than its true error during the outage, the very first fixes after the outage look like outliers and get rejected. Each rejection keeps the estimate un-corrected, so the next fix is rejected too. I wrote a unit test (`gps_return_can_be_gated_out_after_loss`) that reproduces exactly this: an accelerometer bias the basic filter doesn't model, 20 s without GPS, and the returning fix is rejected by the 3-σ gate.

The variance ramp. After a stale gap (no fix for 1.5 s), returning fixes are given inflated measurement noise $R' = s \cdot R$, where s decays quadratically from an initial multiplier to 1 over a ramp window:

```
s(t) = 1 + (s0 − 1) · (1 − p)²,     p = clamp((t − t_return) / T, 0, 1)
```

One correctness detail matters here. The inflated R' has to be used consistently in the innovation covariance, the Cholesky solve for the Kalman gain, and the Joseph-form covariance update ($P^{+} = (I - KH)\,P\,(I - KH)^{T} + K\,R'\,K^{T}$), while the outlier gate still uses the receiver's nominal R. If you inflate R for the gain but leave the nominal value in the Joseph term, the filter reports less uncertainty than it actually has. I changed all three places together.

Tuning against a stated priority. The original ramp (1000x over 3 s) trades accuracy for smoothness: it shrinks the biggest correction jump but keeps the estimate wrong for longer. On the same seed, queue settings, and input stream:

Recovery setting	Return RMSE (50–53 s)	Post-recovery RMSE	Largest return step	Final error
1000x over 1 s (prior)	1.078 m	0.108 m	3.633 m	0.0300 m
800x over 0.5 s	0.885 m	0.106 m	3.912 m	0.0303 m
1.5x over 0.5 s (chosen)	**0.260 m**	0.104 m	5.776 m	0.0311 m
Abrupt return (control)	0.252 m	0.103 m	5.777 m	0.0315 m

The chosen setting cut return-window error **75.9%** versus the prior ramp and lands within 0.008 m of the abrupt control. I documented the cost too: it gives back the smaller correction jump. That's a deliberate trade for accuracy, and it's a one-seed result, so I labeled it that way.

4. The hard part: a drift-invariant check for returning GPS

A variance ramp treats every returning fix the same. What you actually want is to trust a good fix immediately and stay skeptical of a bad one (multipath, a jump, a spoofed position). The obvious test, "is the fix close to my estimate?", is useless right after an outage, because the estimate has drifted by meters. A perfectly good fix looks just as far off as a bad one.

My adaptive check compares *motion* instead of *position*. For two successive returning fixes z_{k-1} and z_k, with filter estimates $\hat{x}_{k-1}$ and the IMU-propagated prediction $\hat{x}_k^-$:

$$d = \| (z_k - z_{k-1}) - (\hat{x}_k^- - \hat{x}_{k-1}) \|$$
$$\text{consistent} \iff d \le 3 \cdot \max\left(\sqrt{\sigma^2_{k-1} + \sigma^2_k},\ 0.25\ \text{m} \right), \quad \text{with } 0 < t_k - t_{k-1} \le \text{stale limit}$$

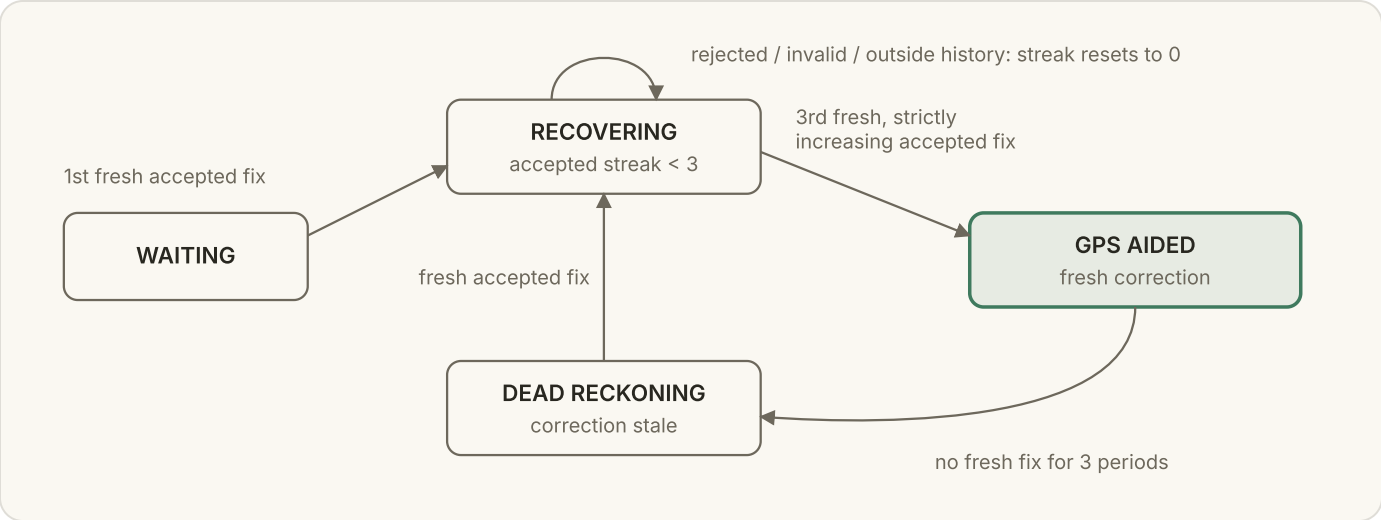
Why this works: dead-reckoning error over a 40-second outage is mostly an accumulated offset that changes slowly. Over the half second between two fixes, that offset is nearly constant, so it cancels when you subtract the two positions. What's left compares how far GPS says the robot moved against how far the IMU says it moved over the same short interval, which the IMU measures well even when its absolute position is meters off. The threshold is 3-σ of the difference of two independent fixes (variances add), with a 0.25 m floor so a too-confident receiver can't make the test impossible to pass.

- A consistent pair returns GPS to nominal weight at once.
- A mismatch holds a 5x variance multiplier and keeps checking.
- The first fix after the gap, which has nothing to pair with yet, gets the short ramp.
- Only applied fixes advance the pair. A rejected fix resets it, so one bad fix can't vouch for the next.

Measured effect (queue-matched, same seed) versus no ramp: overall RMSE −1.8%, outage-window RMSE −1.9%, final error −46%, at a cost of +5.2% return-window RMSE. In the queue-free end-to-end test it cut final error from 0.0418 m to 0.0263 m. The honest caveat, which is also in the repo: the mismatch branch still needs a controlled GPS-jump experiment before I'd call it robust.

5. Knowing when to trust the estimate: the GPS-health monitor

A filter will happily report a position whether or not it has been corrected recently. Operators need to know which situation they are in, so I built a monitor that replays every GPS decision (accepted, rejected, invalid, outside the filter's history buffer) in arrival order and reconstructs a four-state machine. It observes the filter and never changes its tuning or state.



- **Stale** means no fresh correction for three nominal GPS periods (1.5 s at 2 Hz).
- **Re-entering GPS AIDED** requires three fresh accepted fixes with strictly increasing measurement times. A rejected, invalid, or outside-history fix resets the streak to zero, so a burst of bad or replayed packets can't fake recovery.
- **Two different ages.** Receipt age is "how long since any packet arrived"; correction age is "how old is the newest measurement the filter accepted." A delayed or out-of-order packet updates the first but not the second, so late data never looks like fresh data.
- **Output:** every fix decision with its normalized residual, health samples over time, and a notification with recommended operator action only on state changes, saved to `gps-health.json` and shown live in the Rerun dashboard.

The unit test walks a 60-second run with a 40-second outage and a rejected first return fix, and asserts the exact sequence: dead reckoning at 49 s, still dead reckoning when the rejected fix arrives at 50 s (receipt age 0, correction age 41 s), recovering at 51–52 s, aided at 53 s, and exactly five notifications.

How I kept it honest

- **Regression baselines:** every experiment's metrics live in a fixtures file. Tests fail if a change moves them, so improvements are always deliberate and reviewed.
- **Paired comparisons:** policy comparisons use the same seeds, service times, and input streams, so differences come from the policy and not the noise.
- **Window metrics, not just totals:** outage [10, 50) s, return [50, 53) s, and post-recovery [53, 60) s are scored separately, because aggregate RMSE hides exactly the moments that matter.
- **Tooling:** `analyze-runs.py` exports saved-run metrics and deltas against a reference run to CSV, with its own tests.
- **Stated limits:** recovery results are single-seed and simulated. My advisor's next request is to characterize real open-source sensor data, since simulated noise is cleaner than reality.

What I'd do next

- Multi-seed recovery sweeps with confidence intervals, plus outages of different lengths and motion profiles.
- Inject controlled GPS jumps and multipath to test the mismatch branch of the consistency check.
- Replace drop-and-compensate with true out-of-sequence measurement handling in the EKF (retrodiction from a state history buffer).
- Re-run the queue and recovery experiments on real GNSS/IMU logs, such as UrbanNav, to see how much of this survives real noise.

PYTHON · OPENCV · CLIP · MEDIAPIPE · CLAUDE API · NOV 2026

TrueTake

A pipeline that decides which recorded robot-training clips a lab should accept. Its core is recovering the clock offset between a phone's video and its gyroscope, which also proves the footage is genuine.

18 axis and sign pairings searched, then locked per device	**±2 s** lag search for the video-to-sensor clock offset	**5 s / 1 s** sliding windows that localize splices	**3 layers** quality, verification, value; LLM only when unsure

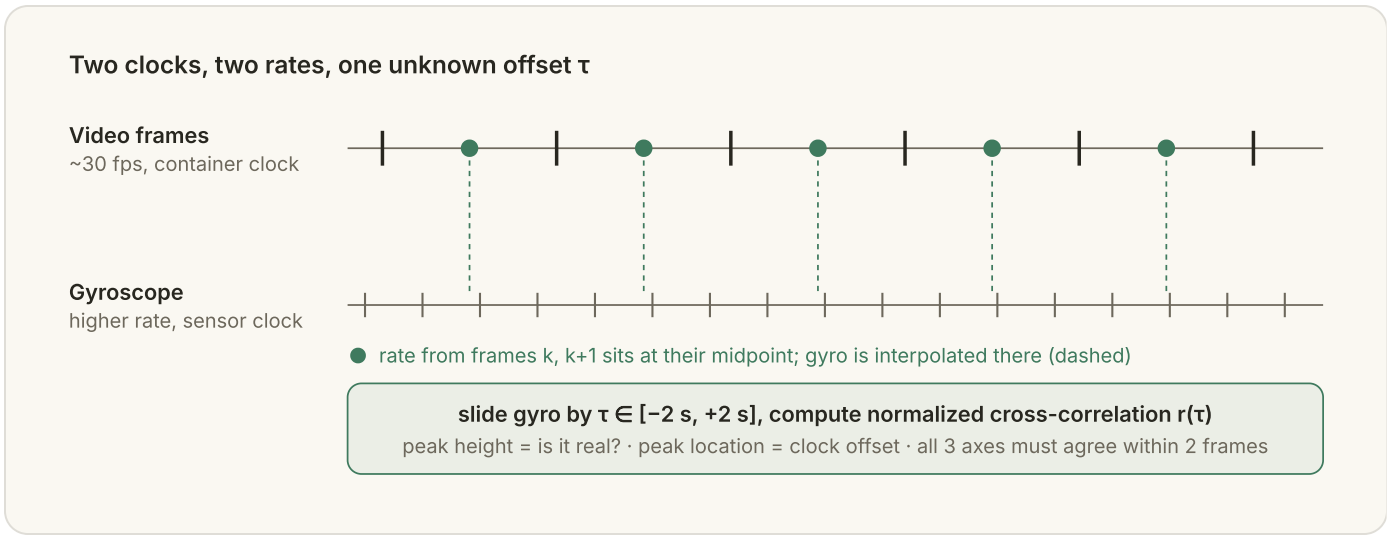
The problem

Robotics labs pay people to record demonstrations, such as folding a shirt on video, as training data. Many clips turn out to be unusable: badly framed, idle, duplicated, or not genuine (a re-filmed screen, a borrowed sensor file, a spliced segment). TrueTake is a pipeline that decides which clips to accept and explains every decision with timestamped reasons and usable minutes per clip. It is built as a cheap-first cascade. Fast deterministic checks decide most clips, and a vision-language model (Claude) is called only for the cases those checks can't settle.

- **Layer 1, quality:** resolution, frame rate, orientation, blur, idle time, and whether the task is in frame. Cheap CLIP checks run first; Claude judges only the uncertain middle and has to cite frames as evidence.
- **Layer 2, verification:** does the video's motion match the phone's own gyroscope? (the main section below)
- **Layer 3, value:** how much new behavior a clip adds compared with the clips already kept.

Verification is a clock-synchronization problem

A phone records video and motion sensors at the same time, but into two different files, on two different clocks, at two different rates. If the footage is real and unedited, the way the picture rotates must match the way the phone rotated. Testing that means first answering a harder question: **what is the time offset between the two files?** Labs need that offset anyway to align video with sensor data, so one computation does double duty. It gives a match score (is it genuine?) and a clock offset (how to sync it).



Step by step

1. Camera rotation from pixels. For each pair of consecutive frames: grayscale, downscale to 320 px wide, pick about 200 corners (`goodFeaturesToTrack`), track them into the next frame with pyramidal Lucas–Kanade optical flow, and fit a rotation-plus-translation (partial affine) model with RANSAC so hands and moving cloth are rejected as outliers. That yields horizontal shift *dx*, vertical shift *dy*, and in-image rotation *dθ*. Dividing by the inter-frame time gives three rate signals: pan, tilt, and roll.

2. Put the gyro on the video's time grid, at the right instant. A finite difference between frames *k* and *k+1* estimates the rate at their *midpoint*, not at either frame. So each gyro axis is interpolated onto frame-pair midpoints. Sampling at frame times instead would bake in a systematic half-frame error (about 17 ms at 30 fps) that the cross-correlation would then report as part of the clock offset.

3. Clean both signals. A 0.2 s moving average suppresses tracking jitter, and subtracting each signal's mean removes constant terms such as gyro bias or a slow steady pan.

4. Figure out which axis is which. How gyro axes map to pan, tilt, and roll depends on how the phone was held. For each video signal, the pipeline tries all three gyro axes with both signs (18 candidate pairings) and keeps the best-correlating one. The mapping is learned once per device and orientation on a known-good clip and then locked, so an attacker can't benefit from the pipeline searching harder on a bad clip.

5. Cross-correlate over lag. For each matched pair, compute the normalized cross-correlation at every lag from −2 s to +2 s:

$$r(\tau) = \Sigma_t\,(v_t - \bar{v})(g_{t+\tau} - \bar{g}) \;\Big/\; \left(\sqrt{\Sigma_t (v_t - \bar{v})^2}\cdot\sqrt{\Sigma_t (g_{t+\tau} - \bar{g})^2}\right)$$

The peak height is the match score; the lag at the peak is the clock offset. Because Pearson correlation ignores scale, you never need the camera's focal length. For small rotations, image shift is about focal length × angular rate × Δt, so the video signal is a scaled copy of the gyro signal, and the unknown scale drops out.

6. Make the three axes vote. Pan, tilt, and roll must all peak at nearly the same lag (within 2 frames). One axis can produce a spurious peak by chance; three independent axes agreeing on the same offset is much harder to fake.

7. Look inside the clip. The same correlation runs on 5-second windows sliding by 1 second. A window where the match collapses inside an otherwise matching clip marks a splice or an inserted segment, and the report gives its timestamps.

8. Decide without false accusations. If the gyro barely moved (its spread is below a small threshold), the result is "not enough motion to verify," not a failure, because a still camera proves nothing either way. Otherwise a clip passes when its score clears a threshold set on training clips (the level that keeps 95% of genuine clips), and offsets over 1 second are flagged.

What each attack looks like to the math

Attack	Signature
Sensor file from a different clip	Low peak at every lag
Sensor file with its clock shifted	Strong peak, but at the wrong offset
Screen re-filmed from a tripod	Video shows motion, gyro is flat: no match
Spliced or inserted segment	Match collapses in specific windows
Stock or generated video with made-up sensor data	Low peak
Screen re-filmed with a hand-held phone	Partial match because the phone really moved; a known gap I document rather than hide

Practical limits I designed around: phone video stabilization removes the very shake being matched, so it must be off; close-up hands and cloth add motion that isn't camera rotation, so thresholds come from real tabletop clips; heavy motion blur loses tracked corners, so those frames count as inconclusive. Tracking 200 points at 320 px is light enough that a 30-second clip takes seconds on a laptop.

Value: which clips actually teach a robot something new

Only clips that pass quality and verification are scored, so "unusual" can't just mean "broken." Each clip gets three views, each with its own distance:

- **Scene:** CLIP ViT-B/32 embeddings of 8 frames, averaged; distance is 1 − cosine similarity.
- **Hand motion:** MediaPipe wrist, thumb-tip, and index-tip tracks, normalized to frame size and resampled to 64 steps; distance is dynamic time warping.
- **Strategy:** Claude labels the fold as a sequence from a fixed step list (smooth, sleeves-in, fold-half, fold-thirds, roll, drop, recover, and so on); distance is normalized edit distance.

Each view's distances are divided by that view's median and averaged. Ranking is greedy farthest-point selection: start from the most typical clip, then repeatedly add the clip farthest from everything

already chosen. A clip's value is its distance to the nearest chosen clip when it's added, and a clip whose nearest neighbor is too close is marked redundant, with its twin named in the report.

Evaluation and how I built it

Every check was scored against hand labels on held-out clips plus synthetic attacks generated for each attack type above, reporting attacks caught, genuine clips wrongly flagged, and inconclusive clips, compared with a random baseline. Per-clip model calls and frames processed are logged so cost and failure modes can be traced. I used Claude Code as a coding agent, one module per prompt, and reviewed and tested every generated file on real clips before committing it. The two bugs it is easiest to get wrong, mixed timestamp units (ns, ms, s) and degrees versus radians, are checked by printing duration and gyro range on load and by plotting both signals together in Rerun before trusting any score.

What I'd push further

- Sub-frame clock offset by fitting a parabola through the correlation peak and its neighbors, instead of snapping to the nearest midpoint sample.
- Rolling-shutter correction: each image row is captured at a slightly different time, which skews rotation estimates during fast motion.
- Read gyro data embedded in action-camera files (GoPro and similar) so verification works beyond phones.